

Instagram Data Viewer — MCP Integration Build Plan

Prepared by Claude · 2026-09-16 · Updated 2026-09-17 — all phases complete

Status: built and in daily use. This document originally laid out the plan before any of it existed; it's now kept as the as-built record of what was actually built, including a few things that turned out differently than planned. For day-to-day usage instructions, see `INSTAGRAM_MCP_GUIDE.md`.

Overview

This plan extends the existing `instagram_service` package (auth, media, comments, insights) with the write operations needed for Claude to manage the Instagram account conversationally: publishing posts, replying to and moderating comments, and reading/sending direct messages. The new capabilities are exposed to Claude through a local MCP (Model Context Protocol) server, wrapping the same service modules already used by the Flask dashboard. This project's scope is now MCP-only — the Instagram account is managed entirely through a Claude agent talking to the MCP server, with no dashboard UI work planned.

Recommended Approach

Build a local MCP server on top of `instagram_service` rather than a separate client. The write-side modules are added first, tested with the existing pytest suite, then wrapped as MCP tools. A companion Skill captures workflow conventions once the tools exist. Meta's own Social Technologies MCP (developer-assistance tooling) was considered and set aside for now — it does not expose account-actions and is not required for this build.

Meta App Permissions Required

The project's access token uses the IGAA prefix (Instagram API with Instagram Login). Meta migrated this flow's scope names; the app must request the new names below:

Capability	Scope needed
Read profile / media	<code>instagram_business_basic</code>
Publish posts / reels / carousels	<code>instagram_business_content_publish</code>
Read / reply to comments	<code>instagram_business_manage_comments</code>
Read / send DMs	<code>instagram_business_manage_messages</code>

Old scope names (`business_basic`, `business_content_publish`, `business_manage_comments`, `business_manage_messages`) are deprecated.

Constraints to Design Around

- Publishing is capped on a rolling 24-hour window — check `GET /{ig-user-id}/content_publishing_limit` live rather than hardcoding a number (Meta's own docs are inconsistent between 25 and 50; this account's actual quota is 100/24h).

- DMs can only be freely sent inside the 24-hour standard messaging window opened by the recipient's last message; automated sends are also throttled (~200/hr/account).
- General call volume follows Meta's Business Use Case formula (roughly 4800 x recent impressions per 24h) — low-engagement accounts throttle fast.
- Any use beyond the developer's own linked account likely needs Advanced Access via Meta App Review.
- **New finding:** `create_media_container/preview_post` need a public HTTPS URL that Meta's servers fetch directly. Google Drive links — even a corrected direct-image URL (`lh3.googleusercontent.com/d/...`) — fail against Meta's fetcher in practice, likely because Google's CDN treats Meta's origin IPs differently. Imgur direct links (`i.imgur.com/HASH.jpeg`) work reliably. See the guide's "Posting an image" section.

Phase 0 — Meta App Configuration (Complete)

- Confirm the app's Instagram use case is set to "API setup with Instagram Login."
- Add the four scopes from the permissions table above.
- Refresh the long-lived token so it carries the new scopes.
- Confirm via `diagnose_env.py` that the refreshed token still parses with the existing IGAA-prefix logic in `auth.py`.

Done by Krish directly in the Meta Developer dashboard; verified afterward by successfully calling scope-gated endpoints (`content_publishing_limit`, `conversations`) with the refreshed token.

Phase 1 — `instagram_service/publishing.py` (Complete)

- `create_media_container(ig_user_id, access_token, media_type, image_url, video_url, caption)` → `POST /{ig-user-id}/media`
- `get_container_status(container_id) / wait_for_container_ready(...)` → poll `status_code` until `FINISHED`, raising `MediaContainerError` on `ERROR` or timeout
- `publish_container(ig_user_id, container_id, access_token)` → `POST /{ig-user-id}/media_publish`
- `get_publishing_limit(ig_user_id, access_token)` → wraps `GET /{ig-user-id}/content_publishing_limit`
- New typed exceptions in `exceptions.py`: `MediaContainerError`, `PublishingQuotaExceededError` (mapped from Graph API code 9007)
- 9 unit tests in `tests/test_publishing.py`, all mocked (no real token/network)

Phase 2 — Extend `instagram_service/comments.py` (Already existed)

Turned out `reply_to_comment`, `delete_comment`, and `set_comment_visibility(hide=True/False)` were already implemented in the pre-existing codebase — no new code needed here, just wrapping them as MCP tools in Phase 4 (`hide_comment` wraps `set_comment_visibility`).

Follow-up fix (2026-09-17): `DEFAULT_COMMENT_FIELDS` was updated to also request the `from` field. Graph API's plain `username` field is only reliably populated for the account's own comments; `from.username` is what's actually populated for every commenter, so `list_comments` now surfaces real usernames for everyone instead of leaving other commenters blank.

Phase 3 — `instagram_service/messaging.py` (Complete)

- `list_conversations(ig_user_id, access_token)` / `get_all_conversations(...)`
- `get_messages(conversation_id, access_token)`
- `send_message(ig_user_id, recipient_id, message, access_token, last_incoming_message_ts=None)`
- `is_within_messaging_window(last_incoming_message_ts)` — accepts a Unix timestamp or Graph API's ISO 8601 string; `send_message` calls this automatically and raises the new `MessagingWindowExpiredError` locally instead of letting Meta reject a stale send

8 unit tests in `tests/test_messaging.py`

Phase 4 — `mcp_server.py` (project root) (Complete)

Built with the Python MCP SDK's classic FastMCP API (pinned to `mcp==1.30.0` — `mcp 2.x` renamed FastMCP to MCPServer with a different interface, so the pin avoids a breaking upgrade). Each tool is a thin wrapper over Phases 1–3 plus the existing `media.py` / `insights.py` modules. All 12 tools implemented exactly as planned:

- `get_account_profile`, `list_media`, `get_media_insights`
- `list_comments`, `reply_to_comment`, `hide_comment`, `delete_comment`
- `preview_post` (builds the container, waits for it to finish, returns status + quota — no publish) / `publish_post` (publishes a previewed container)
- `list_conversations`, `get_messages`, `send_message_reply`

Every write tool is split into a preview/commit pair so Claude confirms before anything is posted, replied to, or sent.

Server naming: the server was originally registered as "instagram", then renamed to **krish-instagram** on 2026-09-17 at Krish's request (cosmetic only — same code, same tools, just a friendlier identifier in `/mcp` and tool names like `mcp__krish-instagram__list_media`). A fully spaced name like "Krish Instagram MCP" isn't possible — Claude Code requires server keys to be letters/digits/hyphens/underscores only, with no separate display-name field.

Phase 5 — Tests (Complete)

- Extended the pytest suite with mocked HTTP responses for `publishing.py` and `messaging.py`, matching the existing `auth/comments/insights` test style.
- Added `tests/test_mcp_server.py` — calls tools the way a real MCP client would (`mcp.call_tool`, not the raw Python functions) to confirm FastMCP's actual exception-wrapping behavior (`ToolError("Error executing tool <name>: <message>")`) produces sensible messages for `TokenExpiredError`, `PublishingQuotaExceededError`, `MediaContainerError`, and `MessagingWindowExpiredError`.
- 45 tests passing, run with `python3 -m pytest` (bare `pytest` causes `ModuleNotFoundError` on this setup).

Phase 6 — Wire into a Claude agent (Complete — CLI only, see note)

- **Claude Code CLI:** registered `mcp_server.py` as a local stdio MCP server via `.mcp.json` in the project root. Works as planned — this is the intended, working path for using this server today.

- **Claude Desktop (this app's "Connectors" UI):** does not support this. The original plan assumed the classic Claude Desktop `claude_desktop_config.json` format with a `command/args` entry for local servers. In practice, this app's Connectors UI ("Add custom connector") only accepts a remote HTTPS MCP server URL — there's no option for a local command. Wiring this server into that UI would require running it over HTTP(S) and exposing it via a public tunnel, which was deliberately not pursued: this server can publish/delete/DM on a real account, and putting it on the public internet for the sake of a convenience CLI already covers wasn't judged worth the added attack surface.

Smoke-tested in this order per the original plan: read tools first (`get_account_profile`, `list_media`, `list_conversations`), then a throwaway comment reply, then a real test post (the "Rainbow" post, via an Imgur-hosted image) — all successful.

Phase 7 — Companion Skill (Complete — expanded beyond original scope)

`.claude/skills/instagram-mcp-workflow/SKILL.md` covers everything originally planned — caption/hashtag conventions, reply vs. hide vs. delete, DM tone, and the hard confirmation rule before any `publish_*/send_*/delete_*` call — plus two rules added after real usage surfaced gaps:

- **Scope of use:** tools may only be called in direct response to something explicitly asked in the current conversation — no proactive or speculative calls, and no scheduled/background automation of any kind exists in this project.
- **Real preview, not just metadata:** `preview_post`'s return value is only container status/quota JSON, not the actual media — the skill now requires actually showing the user the image/video itself (not just its URL) plus the caption quoted verbatim, since showing only the JSON isn't a real preview to approve on.